SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Machine Learning

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO4 – Precision	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	D. Sri Anjaneya	**Reg. No.:**	192425219
Section / Batch:	AI & ML / 2024	**Date:**	13/08/2026

Code of Conduct

I _D. Sri Anjaneya / 192425219_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __D. Sri Anjaneya__

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3

Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing.
Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques.
Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames.
Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing.
Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

1. CFG Trees vs Dependency Parsing

Introduction

Both **Context-Free Grammar (CFG)** and **Dependency Parsing** represent sentence structure, but they use different approaches.

- **CFG** represents sentences using hierarchical **constituent structures** such as NP (Noun Phrase) and VP (Verb Phrase).
- **Dependency parsing** represents relationships directly between individual words using **head–dependent relationships**.

Example

Sentence:

"The student reads a book."

CFG Representation

A CFG represents the sentence hierarchically:

S
├── NP
│ ├── Det → The
│ └── N → student
└── VP
├── V → reads
└── NP
├── Det → a
└── N → book

The sentence is divided into constituents such as **NP** and **VP**.

Dependency Representation

Dependency parsing focuses on word-to-word relationships:

reads
├── student → subject
└── book → object

Here, **reads** is the main head, while **student** and **book** depend on it.

Comparison

Aspect	CFG Trees	Dependency Parsing
Basic unit	Constituents/Phrases	Words
Structure	Hierarchical tree	Head–dependent tree
Main focus	Phrase structure	Word relationships
Subject/object relation	Indirectly represented	Directly represented
Long-distance relationships	Can be represented	Often represented directly
Grammar checking	Very useful	Useful
Semantic relationships	Less direct	More direct
Applications	Parsing, grammar analysis	Information extraction, NLP, relation analysis

For **capturing relationships between words**, dependency parsing is generally more effective.

For example:

“The doctor prescribed medicine to the patient.”

Dependency parsing can directly represent:

doctor → prescribed → medicine

and

prescribed → patient

This makes it useful for information extraction and semantic analysis.

However, CFG remains highly useful when the primary requirement is **constituent structure and formal grammar checking**.

Justification

Therefore, the choice depends on the application:

- **Grammar/constituent analysis → CFG**
- **Word relationships → Dependency Parsing**

For modern NLP systems that need to understand relationships between words, **dependency parsing is generally the better choice**.

2. Top-Down Parsing vs Earley Parsing

Introduction

Top-down parsing starts from the grammar's start symbol and attempts to generate a structure matching the input. **Earley parsing** processes the input while maintaining partial parsing states and can handle general CFGs.

Top-Down Parsing

The process is:

Start Symbol → Grammar Rules → Input Matching

For example:

The parser first expands , then and , and continues until it matches the input.

Earley Parsing

Earley parsing uses three major operations:

1. **Prediction** – predicts possible grammar rules.
2. **Scanning** – matches grammar symbols with input words.
3. **Completion** – completes rules when their required components are recognized.

It stores partial results to avoid unnecessary repeated computation.

Comparison

Aspect	**Top-Down Parsing**	**Earley Parsing**
Starting point	Start symbol	Input + grammar states
Ambiguity	Can require backtracking	Handles ambiguity systematically
Incomplete input	Less suitable	Better suited
Left recursion	Can cause problems	Supported
Backtracking	Often required	Reduced
Long/complex sentences	Can become inefficient	Better for general CFGs
Dynamic input	Limited	More flexible
Grammar coverage	Depends on implementation	General CFG

Example of Incomplete Input

Consider:

“Book a flight to…”

A real-time system may receive this before the user finishes speaking.

Earley parsing can maintain a **partial parse state** and continue processing when additional words arrive.

Critical Analysis

Top-down parsing is simple and can be efficient for small, predictable grammars. However, real-time NLP systems frequently encounter:

- Incomplete sentences.
- Ambiguous structures.
- Speech recognition errors.
- Long sentences.
- User corrections.

Earley parsing is better suited to these conditions because it can maintain multiple possible states and process incomplete structures.

Recommendation

For dynamic real-time applications, **Earley parsing is generally more suitable**, particularly when the grammar is complex and ambiguity is common.

A practical system could use:

Speech/Input → Earley Parser → Probabilistic Ranking → Semantic Interpretation → Action

Conclusion

Top-down parsing is simpler, but Earley parsing provides greater flexibility for **ambiguous, incomplete and dynamically arriving input**. Therefore, Earley parsing is the stronger choice for general real-time NLP applications.

3. CFG vs PCFG vs Neural Parsing for Ambiguity

Consider:

“She saw the man with a telescope.”

This sentence has **prepositional phrase attachment ambiguity**.

Interpretation 1

She used a telescope to see the man.

[She saw the man] [with a telescope]

Interpretation 2

The man had a telescope.

She saw [the man [with a telescope]]

The parser must determine which interpretation is more likely.

CFG

A CFG can generate both valid structures if the grammar allows both attachments.

However, basic CFG generally treats the alternatives as grammatical possibilities without providing a strong mechanism for deciding which meaning is intended.

Advantage

- Simple and formal.
- Good for representing syntactic structures.

Limitation

- Does not naturally rank competing interpretations.

PCFG

A **Probabilistic Context-Free Grammar** associates probabilities with grammar rules.

For example:

The parser can select the higher-probability structure.

Advantage

PCFG uses statistical information to resolve ambiguity.

Limitation

Its decisions depend strongly on the quality and coverage of the training grammar and may not capture rich contextual meaning.

Neural Parsing

Neural parsers use neural networks and contextual representations to determine the most likely syntactic structure.

They can consider:

- Surrounding words.
- Sentence context.
- Word meaning.
- Learned linguistic patterns.
- Large training datasets.

For example, the model can learn from previous examples that phrases such as **"with a telescope"** are frequently associated with the action of seeing.

Comparison

Aspect	**CFG**	**PCFG**	**Neural Parsing**
Structure	Rule-based	Probabilistic rule-based	Neural/contextual
Ambiguity	Generates alternatives	Ranks alternatives	Uses learned context
Context understanding	Low	Moderate	High
Data requirement	Low	Training corpus for probabilities	Usually large datasets
Interpretability	High	High/Moderate	Lower
Flexibility	Limited	Better	High
Real-world NLP	Basic	Useful	Strong

For **real-world applications**, neural parsing is generally the most powerful because it can use contextual information and learned representations to resolve ambiguity.

However, a **hybrid approach** is often preferable in industry:

Grammar Rules + PCFG + Neural Contextual Model

CFG provides structural constraints, PCFG provides probabilistic ranking, and neural models provide rich contextual understanding.

Conclusion

CFG is best for formal syntactic structure, PCFG improves ambiguity resolution through probabilities, and neural parsing provides stronger contextual understanding. Therefore, **neural or hybrid neural-probabilistic parsing is generally most suitable for modern real-world NLP systems**.

4. Feature Structures vs Subcategorization Frames

Introduction

Both **Feature Structures** and **Subcategorization Frames** help NLP systems understand grammatical relationships, but they solve different problems.

- **Feature structures** represent grammatical properties such as number, person, tense and gender.
- **Subcategorization frames** specify what types of arguments a verb requires.

Feature Structures

A feature structure can represent:

Person = 3

Number = Singular

Tense = Present

For example:

The doctor recommends treatment.

The subject is singular:

The verb should also be singular:

Therefore:

doctor + recommends → correct agreement.

But:

doctor + recommend → agreement error.

Advantages

- Handles subject–verb agreement.
- Represents grammatical features.
- Supports feature unification.
- Useful for enforcing syntactic consistency.

Subcategorization Frames

A subcategorization frame specifies the arguments required by a verb.

For example:

“Prescribe”

Example:

The doctor prescribed medicine to the patient.

The verb **prescribe** expects:

- Agent → Doctor

- Theme → Medicine
- Recipient → Patient

“Sleep”

It normally requires only a subject.

Thus, different verbs have different argument structures.

Comparison

Aspect	**Feature Structures**	**Subcategorization Frames**
Main purpose	Represent grammatical features	Represent verb argument requirements
Handles agreement	Excellent	Limited
Handles verb arguments	Limited	Excellent
Examples	Number, person, tense	Subject, object, complement
Main use	Grammatical correctness	Verb–argument relationships
Prevents	Agreement errors	Incorrect argument structures

For **subject–verb agreement**, feature structures provide better support.

For **verb argument structures**, subcategorization frames are more appropriate.

Therefore, neither completely replaces the other.

Best Solution

A robust grammar system should combine both:

Feature Structures → Agreement

Subcategorization Frames → Verb Arguments

For example:

The doctors prescribed medicines to patients.

Feature structures verify:

doctors → prescribed

Subcategorization verifies:

prescribe → doctor + medicine + patient

Conclusion

Feature structures are better for **grammatical agreement**, while subcategorization frames are better for **verb argument structure**. A combined system provides more complete grammatical correctness than either technique alone.

5. Transition-Based vs Graph-Based Dependency Parsing

Introduction

Both **transition-based parsing** and **graph-based parsing** are used for dependency parsing, but they use different strategies.

- **Transition-based parsing** constructs the dependency tree through a sequence of parsing decisions.
- **Graph-based parsing** considers possible dependency relationships and searches for the best complete tree.

Transition-Based Parsing

Transition-based parsers maintain a parsing configuration such as:

Stack + Buffer + Dependency Relations

The parser repeatedly chooses actions such as:

- **SHIFT**
- **REDUCE**
- **LEFT-ARC**
- **RIGHT-ARC**

For example:

Input → Stack/Buffer → Parsing Decisions → Dependency Tree

Advantages

- Very fast.
- Usually linear-time or close to linear-time in common implementations.
- Requires relatively low memory.
- Suitable for real-time applications.
- Scales well to large datasets.

Limitation

A wrong early decision can affect later decisions because the parser builds the tree incrementally.

Graph-Based Parsing

Graph-based parsing considers dependency arcs between words.

The parser searches for the dependency tree with the highest total score.

Conceptually:

It evaluates the complete structure rather than committing to every decision immediately.

Advantages

- Global optimization.
- Can consider relationships across the entire sentence.
- Less dependent on irreversible early decisions.
- Often produces strong parsing accuracy.

Limitations

- Generally more computationally expensive.
- Requires more memory.
- Can be slower than transition-based parsing for very large datasets.

Comparison

Feature	**Transition-Based**	**Graph-Based**
Main approach	Sequence of actions	Search for best complete tree
Decision-making	Local/step-by-step	Global
Speed	Very high	Generally lower
Memory	Lower	Higher
Error propagation	More likely	Less dependent on early local decisions
Accuracy	Often strong	Often strong, especially with global scoring
Large-scale processing	Excellent	More computationally demanding
Real-time applications	Highly suitable	Less suitable when latency is strict

Example

For a large dataset containing millions of sentences, a transition-based parser can process sentences quickly because it makes a sequence of relatively inexpensive decisions.

A graph-based parser may examine many possible dependency relationships before selecting the best tree, providing global reasoning at greater computational cost.

Recommendation for Large-Scale Applications

For **large-scale NLP systems where speed and scalability are major requirements**, **transition-based parsing is generally the better choice**.

It provides:

1. High processing speed.
2. Lower memory requirements.
3. Good scalability.

4. Suitability for real-time applications.
5. Efficient processing of large datasets.

However, if maximum parsing accuracy and global structural decisions are more important than latency, **graph-based parsing** may be preferred.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____